

Back-end Frameworks

Conectando ao banco de dados

Profº: Joseph Donald

Contatos:

 www.josephdonald.wordpress.com

 @josephdonald

 030106382@prof.uninassau.edu.br

“O sucesso consiste em ir de fracasso em fracasso sem perder o entusiasmo.”
Winston Churchill



- A biblioteca PyMySQL é uma implementação pura em Python do protocolo cliente MySQL.
- Ela permite que os desenvolvedores se conectem a um banco de dados MySQL e executem consultas e operações diretamente do Python, sem a necessidade de um driver adicional ou extensão.
- Para instalar o PyMySQL você deve executar o seguinte comando:

```
pip install PyMySQL
```



- Dentro do código você vai importar a lib no início do código utilizando:

```
import pymysql
```

- Dentro do código você instanciará a conexão da lib em uma variável:

```
db = pymysql.connect(host='localhost', user='root', passwd='', database='banco_funcionarios')
```



		banco_funcionarios funcionario
	id_func : int(11)	
	nome : varchar(100)	
	telefone : varchar(15)	



- Agora vamos realizar a conexão no banco utilizando os seguintes comandos:

```
Lib > main.py > ...
1  from flask import Flask
2  import pymysql
3
4  app = Flask(__name__)
5
6  db_host = 'localhost'
7  db_user = 'root'
8  db_password = ''
9  db_name = 'banco_funcionarios'
10
11 def conectDB():
12
13     db = pymysql.connect(host=db_host, user=db_user, passwd=db_password, database=db_name)
14
15     print("TESTE DE CONEXAO")
16     print(db.host_info)
17
18
19 # FUNÇÃO PRINCIPAL
20 if __name__ == '__main__':
21     conectDB()
22     app.run(debug=True, port=5000)
23
```



- Além da conexão, é possível realizar quaisquer transações (ou manipulações) no banco de dados através da lib PyMySQL, utilizando os mesmos comandos da linguagem SQL, como:
 - ✓ INSERT
 - ✓ SELECT
 - ✓ UPDATE
 - ✓ DELETE
 - ✓ DROP
 - ✓ CREATE e etc...



Consulta de dados

- Com o PyMySQL podemos enviar consultas SQL para o banco de dados. Isso inclui: seleção de dados, filtragem, ordenação e outras operações relacionadas a leitura. Estes são alguns métodos utilizados para a Consulta dos dados (Read - Leitura):
 - ✓ **cursor.execute(sql)**: Para ler dados do banco de dados, você usará o método **execute** do cursor para executar uma instrução SQL de seleção.
 - ✓ **cursor.fetchone()**: Após executar uma consulta de seleção, você usará **fetchone()** para recuperar a primeira linha de resultados.
 - ✓ **cursor.fetchall()**: Se você espera múltiplas linhas de resultados, pode usar **fetchall()** para recuperar todas as linhas de resultados de uma só vez.



UNINASSAU

FetchOne

```
main.py x index.html
venvaulapymysql > Lib > main.py > index
1 from flask import Flask, render_template
2 import pymysql
3
4 app = Flask(__name__)
5
6 @app.route('/')
7 def index():
8
9     banco = pymysql.connect(host='localhost', user='root', passwd='', database='bancoexemplo')
10    cursor = banco.cursor()
11    sql = "SELECT * FROM funcionario;"
12    cursor.execute(sql)
13
14    resultadoFetchOne = cursor.fetchone()
15    print("RESULTADO DO FETCHONE")
16    print(resultadoFetchOne)
17
18    return render_template('index.html', resultado = resultadoFetchOne)
19
20 if __name__ == '__main__':
21    app.run(debug=True)
```

TERMINAL PORTS PROBLEMS OUTPUT DEBUG CONSOLE

○ RESULTADO DO FETCHONE
(1163, 'Cláudia')
127.0.0.1 - - [01/May/2024 07:42:55] "GET / HTTP/1.1" 200 -

Exemplo de Consulta com PyMySQL

Profº: Joseph Donald

FetchAll

```
main.py x index.html
venvaulapymysql > Lib > main.py > index
1 from flask import Flask, render_template
2 import pymysql
3
4 app = Flask(__name__)
5
6 @app.route('/')
7 def index():
8
9     banco = pymysql.connect(host='localhost', user='root', passwd='', database='bancoexemplo')
10    cursor = banco.cursor()
11    sql = "SELECT * FROM funcionario;"
12    cursor.execute(sql)
13
14    resultadoFetchAll = cursor.fetchall()
15    print("RESULTADO DO FETCHALL")
16    print(resultadoFetchAll)
17
18    return render_template('index.html', resultado = resultadoFetchAll)
19
20 if __name__ == '__main__':
21    app.run(debug=True)
```

TERMINAL PORTS PROBLEMS OUTPUT DEBUG CONSOLE

○ RESULTADO DO FETCHALL
((1163, 'Cláudia'), (1164, 'Jorge'), (1165, 'Moacir'), (1167, 'Caio'))
127.0.0.1 - - [01/May/2024 07:42:20] "GET / HTTP/1.1" 200 -



Inserção ou Atualização de dados – INSERT / UPDATE

- Além de consultar dados, o PyMySQL permite inserir e atualizar registros do MySQL. Isso é essencial para criar e atualizar dados no nosso aplicativo Flask.
- Tanto para Inserir dados como para Atualizar dados, utilizaremos o método:
 - ✓ **cursor.execute(sql)**: Para inserir novos dados no banco de dados ou para atualizar dados no banco de dados, você usará o método **execute()** do cursor para executar a instrução SQL de inserção INSERT ou a de atualização UPDATE.
 - ✓ **banco.commit()**: O **commit()** é o método que efetivamente aplica essas mudanças ao banco de dados, fazendo com que elas se tornem permanentes. Quando você chama **commit()**, todas as operações de modificação de dados são confirmadas e tornadas permanentes no banco de dados.



Remoção de dados - DELETE

- E, também de grande importância, o PyMySQL permite remover registros do MySQL. Isso é essencial para remover registros que não são mais necessário no banco.
- Tanto para Remover dados, também utilizaremos os métodos:
 - ✓ **cursor.execute(sql)**: Para excluir dados do banco de dados, você usará o método **execute()** do cursor para executar a instrução SQL de remoção DELETE.
 - ✓ **banco.commit()**: O **commit()** é o método que efetivamente aplica essas mudanças ao banco de dados, fazendo com que elas se tornem permanentes. Quando você chama **commit()**, todas as operações de modificação de dados são confirmadas e tornadas permanentes no banco de dados.

Cadastro de Funcionários

Recarregar lista de funcionários

Pesquisar por ID ou Nome

Nome

Telefone

Salvar

Lista de Funcionários

ID	Nome	Telefone	Ações
7	Daniel	213465	Editar Excluir
10	Donald	2136549	Editar Excluir
11	Joseph	154531345	Editar Excluir



UNINASSAU

Exemplo de inserção e atualização com PyMySQL

Profº: Joseph Donald

Parte 1 Conexão com banco de dados

```
main.py > index
1  from flask import Flask, render_template, request, redirect
2  import pymysql
3
4  app = Flask(__name__)
5
6  def conectDB():
7      db = pymysql.connect(host='localhost', user='root', passwd='', database='banco_funcionarios')
8      return db
9
```

Parte 2 Rota inicial

```
9
10 @app.route('/')
11 def index():
12     bdConectado = conectDB()
13     cursor = bdConectado.cursor()
14     sql = "select * from funcionario;"
15     cursor.execute(sql)
16     resultado = cursor.fetchall()
17     print(resultado)
18     return render_template('index.html', resultDB = resultado)
19
```



UNINASSAU

Exemplo de inserção e atualização com PyMySQL

Profº: Joseph Donald

Parte 3 Inserção

```
20 @app.route('/cadastrarFuncionario', methods=['POST'])
21 def cadFunc():
22     nomeRecebido = request.form['nome']
23     telefoneRecebido = request.form['telefone']
24     bdConectado = conectDB()
25     cursor = bdConectado.cursor()
26     sql = f"INSERT INTO funcionario(nome, telefone)values('{nomeRecebido}', '{telefoneRecebido}');"
27     cursor.execute(sql)
28     bdConectado.commit()
29     bdConectado.close()
30     return redirect('/')
```

Parte 4 Atualização

```
31
32 @app.route("/atualizarfuncionario", methods=['POST'])
33 def atualizaFunc():
34     idRecebido = request.form['id']
35     nomeRecebido = request.form['nome']
36     telefoneRecebido = request.form['telefone']
37     print("#####")
38     print(idRecebido)
39     print(nomeRecebido)
40     print(telefoneRecebido)
41     print("#####")
42     bdConectado = conectDB()
43     cursor = bdConectado.cursor()
44     sql = f"UPDATE funcionario SET nome = '{nomeRecebido}', telefone = '{telefoneRecebido}' WHERE id_func = {idRecebido}"
45     cursor.execute(sql)
46     bdConectado.commit()
47     bdConectado.close()
48     return redirect('/')
49
```



UNINASSAU

Parte 5 - Remoção

```
50 @app.route('/removerFuncionario', methods=['POST'])
51 def deletarFunc():
52     idFunc = request.form['id']
53     bdConectado = conectDB()
54     cursor = bdConectado.cursor()
55     sql = f"DELETE FROM funcionario WHERE id_func = {idFunc}"
56     cursor.execute(sql)
57     bdConectado.commit()
58     bdConectado.close()
59     return redirect('/')
60
```

Parte 6 - Pesquisa funcionário

```
60
61 @app.route('/pesquisarFuncionario', methods=['POST'])
62 def pesquisarFunc():
63     pesquisa = request.form['pesquisa']
64     bdConectado = conectDB()
65     cursor = bdConectado.cursor()
66     sql = f"SELECT * FROM funcionario WHERE id_func = '{pesquisa}' OR nome LIKE '%{pesquisa}%'"
67     cursor.execute(sql)
68     resultado = cursor.fetchall()
69     print(resultado)
70     return render_template('index.html', resultDB = resultado)
71
```

Parte 7 Função MAIN

```
71
72 if __name__ == "__main__":
73     app.run(debug=True)
```

Desenvolva uma aplicação web usando **Flask** que permita **cadastrar, listar, editar e excluir produtos**, armazenando os dados em um banco **MySQL**.

Funcionalidades

- Página principal com **lista de produtos**.
- Formulário para **adicionar** um novo produto.
- Opção para **editar** as informações de um produto existente.
- Opção para **excluir** produtos.

Produtos Cadastrados

Pesquisar:

+ Adicionar Produto

Excluir

ID	Nome	Preço (R\$)	Quantidade	Descrição	Ações
1	Mouse Gamer	150,00	20	Mouse com LED RGB e 6 botões programáveis	Editar
2	Teclado Mecânico	320,00	15	Teclado com switches azuis e retroiluminação	Editar
3	Monitor 24"	899,99	8	Monitor Full HD com painel IPS	Editar
4	Notebook Lenovo	3.500,00	5	Intel i5, 8GB RAM, SSD 256GB	Editar
5	Webcam Full HD	199,00	12	Câmera com microfone embutido e clip de fixação	Editar